

Post-Quantum Cryptographic Primitives for Banking Compliance: Multi-Authority Threshold Signatures, Proxy Re-Encryption, and Zero-Knowledge Regulatory Proofs

Prabakaran Kannan

Research Scholar, Department of Computer Science and Engineering

National Institute of Technology Puducherry

Email: cs23d2005@nitpy.ac.in

Abstract—The advent of cryptographically relevant quantum computers poses an existential threat to the classical public-key infrastructure upon which global banking systems depend. Current SWIFT messaging, multi-party transaction authorization, and regulatory reporting mechanisms rely on RSA and elliptic curve cryptography, both of which are vulnerable to Shor’s algorithm. Simultaneously, the banking sector faces increasingly stringent regulatory mandates—Basel III/IV capital adequacy, PCI-DSS 4.0 payment security, the EU Digital Operational Resilience Act (DORA), and Sixth Anti-Money Laundering Directive (6AMLD)—that demand demonstrable cryptographic assurances without exposing sensitive financial data. This paper presents three post-quantum cryptographic constructions tailored to banking compliance workflows. First, we introduce a *multi-authority threshold signature* scheme built on ML-DSA-65, enforcing tiered quorum policies across nine banking authority roles with department-uniqueness constraints, time-bounded signing windows, and replay protection. Second, we propose a *proxy re-encryption* protocol for SWIFT and ISO 20022 correspondent banking chains, employing ML-KEM-768 for key encapsulation with SHAKE256-derived re-encryption keys, supporting multi-hop message transformation while preserving originator non-repudiation signatures. Third, we design a *zero-knowledge regulatory proof engine* providing six proof types—balance range proofs via bit-decomposition, capital adequacy attestation, AML screening verification, transaction threshold compliance, audit trail integrity via Merkle trees, and data residency proofs—all using SHA3-256 commitments with Fiat–Shamir-transformed challenges. We analyze the security properties of each construction, including threshold unforgeability, IND-CPA security under proxy re-encryption, and the soundness and zero-knowledge properties of the proof engine. Performance evaluation demonstrates combined signature generation under 150 ms for critical-tier transactions, re-encryption latency below 10 ms per correspondent hop, and proof generation times ranging from 1 to 500 ms depending on proof complexity.

Index Terms—Post-quantum cryptography, threshold signatures, proxy re-encryption, zero-knowledge proofs, banking compliance, ML-DSA, ML-KEM, SWIFT, Basel III, regulatory technology

I. INTRODUCTION

The global banking ecosystem processes trillions of dollars in daily transactions through interconnected systems including SWIFT, TARGET2, CHIPS, and CLS, all secured by public-

key cryptographic infrastructure. The SWIFT network alone facilitates over 45 million messages per day across more than 11,000 financial institutions in over 200 countries [?]. These systems rely on RSA-2048, ECDSA, and Diffie–Hellman key exchange—algorithms that Shor’s algorithm renders insecure on a sufficiently powerful quantum computer [?].

The timeline for cryptographically relevant quantum computers remains debated, yet the harvest-now-decrypt-later threat model demands immediate action [?]. Financial messages encrypted today with classical algorithms may be recorded by adversaries and decrypted once quantum capability materializes. For banking transactions with long-duration compliance obligations—some regulatory records must be retained for seven to ten years—this threat is not speculative but operational.

In response, NIST finalized three post-quantum cryptographic standards in 2024: ML-KEM (FIPS 203) for key encapsulation [?], ML-DSA (FIPS 204) for digital signatures [?], and SLH-DSA (FIPS 205) for stateless hash-based signatures. The Bank for International Settlements has issued guidance recommending proactive PQC migration in the financial sector [?]. However, raw algorithm replacement is insufficient; banking systems require composite cryptographic constructions that enforce regulatory policies while transitioning to quantum-resistant primitives.

This paper addresses three critical gaps in the PQC migration landscape for banking:

Multi-Authority Transaction Authorization. High-value banking operations—wire transfers exceeding regulatory thresholds, sanctions overrides, HSM key ceremonies—require authorization from multiple independent parties. Existing multi-signature schemes based on ECDSA or Schnorr signatures [?] must be replaced with post-quantum alternatives that preserve the nuanced quorum policies demanded by banking governance frameworks.

Correspondent Banking Message Routing. The SWIFT correspondent banking model requires messages to traverse intermediary institutions that must transform encrypted payloads without accessing the underlying payment data. Proxy

re-encryption (PRE) [?] provides this capability, but existing PRE schemes rely on bilinear pairings vulnerable to quantum attacks.

Regulatory Compliance Verification. Banks must demonstrate compliance with Basel III/IV capital adequacy ratios [?], AML screening requirements [?], PCI-DSS payment security [?], and DORA operational resilience [?] to regulators. Zero-knowledge proofs [?] enable compliance attestation without revealing sensitive financial data, aligning with GDPR data minimization principles [?].

A. Contributions

This paper makes the following contributions:

- 1) A *multi-authority threshold signature* construction based on ML-DSA-65 with tiered quorum policies supporting five transaction tiers, nine authority roles, department-uniqueness constraints, time-bounded signing windows, and replay protection via request identifiers.
- 2) A *SWIFT proxy re-encryption* protocol using ML-KEM-768/1024 for key encapsulation with SHAKE256-derived re-encryption keys, supporting MT103, MT202, pacs.008, pacs.009, and camt.053 message types across multi-hop correspondent chains with configurable hop limits and 24-hour re-key validity.
- 3) A *zero-knowledge regulatory proof engine* providing six proof types—balance range, capital adequacy, AML screening, transaction threshold, audit integrity, and data residency—using SHA3-256 hash commitments with Fiat-Shamir non-interactive transformation, achieving proof generation in 1–500 ms with 24-hour validity periods.
- 4) Security analysis establishing threshold unforgeability, IND-CPA security for proxy re-encryption, and computational soundness and zero-knowledge properties of the regulatory proof system.
- 5) Performance evaluation demonstrating operational feasibility within banking latency requirements across all three constructions.

B. Paper Organization

Section II provides background on banking security standards, SWIFT architecture, and PQC fundamentals. Section III presents the system model, threat model, and regulatory requirements. Sections IV, V, and VI detail the three cryptographic constructions. Section VII provides security analysis. Section VIII presents performance evaluation results. Section IX surveys related work, and Section X concludes.

II. BACKGROUND

A. Banking Security Standards

Basel III/IV Framework. The Basel Committee on Banking Supervision (BCBS) defines minimum capital requirements that banks must maintain [?], [?]. Key ratios include Common Equity Tier 1 (CET1) at 4.5%, Tier 1 capital at 6%, total capital at 8%, leverage ratio at 3%, Liquidity Coverage Ratio (LCR) at 100%, and Net Stable Funding Ratio (NSFR) at 100%. Banks must demonstrate compliance without revealing

proprietary risk model details. BCBS 239 further mandates effective risk data aggregation and reporting [?].

PCI-DSS 4.0. The Payment Card Industry Data Security Standard version 4.0 [?] requires strong cryptographic controls for cardholder data protection, including authenticated encryption, key management lifecycle controls, and cryptographic agility for algorithm transitions.

DORA. The EU Digital Operational Resilience Act [?] mandates ICT risk management frameworks, incident reporting, digital operational resilience testing, and third-party risk management for financial entities. DORA explicitly requires cryptographic protections that remain effective against emerging threats, including quantum computing [?].

AML/6AMLD. The Sixth Anti-Money Laundering Directive [?] requires financial institutions to perform customer due diligence and screen transactions against sanctions lists including OFAC-SDN, EU Consolidated, and UN Security Council lists. Compliance must be demonstrable to regulators while protecting customer privacy [?].

B. SWIFT Network Architecture

The Society for Worldwide Interbank Financial Telecommunication (SWIFT) network enables standardized financial messaging between institutions [?]. SWIFT messages follow either the legacy MT (Message Type) format or the newer ISO 20022 standard [?]. In correspondent banking, payment messages traverse intermediary banks that facilitate cross-border transfers. A typical MT103 (single customer credit transfer) may pass through one to three intermediary institutions before reaching the beneficiary bank.

The SWIFT Customer Security Programme (CSP) mandates security controls including encrypted communication, message integrity verification, and access control. The migration to ISO 20022 (particularly pacs.008 and pacs.009 messages) introduces structured XML payloads that carry richer data, amplifying the confidentiality requirements for in-transit message handling.

C. Post-Quantum Cryptographic Standards

NIST finalized the first three post-quantum standards in August 2024 [?], [?]:

ML-KEM (FIPS 203). Module-Lattice-Based Key-Encapsulation Mechanism, derived from CRYSTALS-Kyber [?]. Provides IND-CCA2 security at three levels: ML-KEM-512 (NIST Level 1), ML-KEM-768 (Level 3), and ML-KEM-1024 (Level 5). Public key sizes range from 800 to 1,568 bytes; ciphertext sizes from 768 to 1,568 bytes.

ML-DSA (FIPS 204). Module-Lattice-Based Digital Signature Algorithm, derived from CRYSTALS-Dilithium [?]. Provides EUF-CMA security at three levels: ML-DSA-44 (Level 2), ML-DSA-65 (Level 3), and ML-DSA-87 (Level 5). ML-DSA-65 produces 3,293-byte signatures with 1,952-byte public keys.

D. Zero-Knowledge Proof Fundamentals

A zero-knowledge proof system allows a prover to convince a verifier that a statement is true without revealing any

information beyond the statement’s validity [?]. The Fiat–Shamir heuristic [?] transforms interactive proofs into non-interactive ones by replacing the verifier’s random challenge with a hash of the prover’s commitment: $e = H(C\|\text{context})$, where C is the commitment and H is a cryptographic hash function. Using quantum-resistant hash functions (SHA3-256, SHAKE256) ensures the non-interactive proof remains secure against quantum adversaries [?].

III. SYSTEM MODEL

A. Banking Network Architecture

We consider a banking ecosystem comprising the following entities:

- **Financial Institutions (FIs):** Banks participating in the SWIFT network, each possessing ML-KEM and ML-DSA key pairs.
- **Signing Authorities:** Nine designated roles within each institution—Central Bank Liaison, Treasury, Compliance, Risk Management, Legal, Board Member, Internal Audit, IT Security, and Operations—each with independent ML-DSA-65 key pairs.
- **Correspondent Banks:** Intermediary institutions that facilitate cross-border payment routing.
- **Regulators:** Government agencies (central banks, financial conduct authorities) that verify compliance proofs.
- **Auditors:** Internal and external audit functions that verify audit trail integrity.

B. Regulatory Requirements

The system must satisfy the following regulatory constraints simultaneously:

- 1) **Basel III/IV:** Capital adequacy ratios (CET1 $\geq$ 450 bps, Tier 1 $\geq$ 600 bps, Total $\geq$ 800 bps, Leverage $\geq$ 300 bps, LCR $\geq$ 10000 bps, NSFR $\geq$ 10000 bps) must be provable without revealing exact capital figures.
- 2) **PCI-DSS 4.0:** Cardholder data must be encrypted with authenticated encryption; key management must support algorithm agility.
- 3) **DORA:** ICT risk management evidence must be cryptographically verifiable; operational resilience testing must include cryptographic migration scenarios.
- 4) **AML/6AMLD:** Transaction screening against OFAC-SDN, EU Consolidated, and UN-SC lists must be provable without exposing match details.
- 5) **GDPR Art. 25:** Data minimization requires that compliance verification reveals only the minimum information necessary.
- 6) **BCBS 239:** Risk data aggregation and reporting must maintain integrity through cryptographic audit trails.

C. Threat Model

We consider the following adversarial capabilities:

Quantum Adversary. An adversary with access to a cryptographically relevant quantum computer capable of running Shor’s algorithm. This adversary can break RSA, ECDSA,

and ECDH but cannot solve Module-LWE or Module-SIS problems in polynomial time.

Compromised Authority. Up to $t - 1$ of n signing authorities may be compromised, where t is the threshold for the relevant transaction tier. The adversary may obtain private keys of compromised authorities but cannot forge signatures from uncompromised ones.

Honest-but-Curious Intermediary. Correspondent banks faithfully execute the re-encryption protocol but may attempt to learn the plaintext content of messages they process. The PRE scheme must ensure intermediaries gain no information about the payment payload.

Malicious Regulator. A regulator receiving zero-knowledge proofs may attempt to extract information beyond the proven statement. The proof system must be zero-knowledge even against a computationally unbounded verifier for the hash-based commitment scheme.

IV. MULTI-AUTHORITY THRESHOLD SIGNATURES

A. Overview

We present a multi-authority threshold signature scheme that extends ML-DSA-65 with banking-specific governance policies. Unlike generic t -of- n threshold schemes [?], our construction enforces role-based quorum policies, department-uniqueness constraints, and time-bounded signing windows.

B. Authority Roles and Quorum Policies

The scheme defines nine authority roles $\mathcal{R} = \{\text{Central Bank, Treasury, Compliance, Risk Management, Legal, Board Member, Internal Audit, IT Security, Operations}\}$. Each authority A_i is characterized by a tuple $(id_i, role_i, dept_i, pk_i, sk_i)$, where pk_i and sk_i are ML-DSA-65 key pairs.

Transaction authorization follows tiered quorum policies as defined in Table I.

TABLE I
TRANSACTION TIER QUORUM POLICIES

Tier	t/n	Window	Amount	Required Roles
Standard	1/1	5 min	<\$1M	Any
Elevated	2/3	1 hr	\$1M–10M	Treasury
High Value	3/5	2 hr	\$10M–100M	Treas.+Compl.
Critical	4/7	4 hr	>\$100M	Treas.+Compl.+Risk
Sanctions	3/3	24 hr	Any	Compl.+Legal+Risk

C. Construction

1) *Setup:* Let $\mathcal{A} = \{A_1, \dots, A_n\}$ be the set of n registered authorities. Each authority generates an ML-DSA-65 key pair:

$$(pk_i, sk_i) \leftarrow \text{ML-DSA-65.KeyGen()} \quad (1)$$

The group public key is derived by hashing all individual public keys:

$$GPK = \text{SHA3-256} \left(\left\| pk_{\pi(1)} \| pk_{\pi(2)} \| \dots \| pk_{\pi(n)} \right\| \right) \quad (2)$$

where π is a canonical ordering by authority identifier.

2) *Signing Request Initiation*: An initiator creates a signing request $\mathcal{S}$ for transaction data tx :

$$\mathcal{S} = (rid, H_{tx}, Q, t_{init}, t_{deadline}) \quad (3)$$

where $rid \leftarrow \{0, 1\}^{128}$ is a unique request identifier, $H_{tx} = \text{SHA3-256}(tx)$ is the transaction hash, Q is the applicable quorum policy, t_{init} is the initiation timestamp, and $t_{deadline} = t_{init} + Q.window$ is the signing deadline.

3) *Partial Signature Generation*: Authority A_i contributes a partial signature by:

- 1) Verify $t_{current} < t_{deadline}$ (signing window not expired).
- 2) Verify $role_i \in Q.required \cup Q.optional$ (role eligibility).
- 3) Verify $dept_i \notin \{dept_j : A_j \in \mathcal{S}.signers\}$ (department uniqueness).
- 4) Compute the signing input: $m_i = H_{tx} || rid || id_i$.
- 5) Generate partial signature: $\sigma_i \leftarrow \text{ML-DSA-65}.Sign(sk_i, m_i)$.

Each partial signature σ_i is 3,293 bytes (the standard ML-DSA-65 signature size).

4) *Quorum Verification*: Before combination, the system verifies:

$$|\mathcal{S}.signers| \geq Q.threshold \wedge Q.required \subseteq \{role_i : A_i \in \mathcal{S}.signers\} \quad (4)$$

5) *Signature Combination*: Given t valid partial signatures $\{\sigma_1, \dots, \sigma_t\}$ from authorities with distinct departments, the combined signature is:

$$\Sigma = \text{SHAKE256}(\sigma_{\pi(1)} || \dots || \sigma_{\pi(t)} || H_{tx} || \text{"multi-auth-combination"}) \quad (5)$$

where π is a canonical ordering by authority identifier. The combined signature is 128 bytes, providing a compact attestation regardless of the number of contributing authorities.

6) *Verification*: Verification of a combined signature requires:

- 1) Retrieve the individual partial signatures $\sigma_1, \dots, \sigma_t$ and corresponding public keys.
- 2) For each i , verify $\text{ML-DSA-65}.Verify(pk_i, m_i, \sigma_i) = 1$.
- 3) Recompute Σ' from the verified partial signatures.
- 4) Accept if and only if $\Sigma' = \Sigma$.

D. Department Constraint Enforcement

The department-uniqueness constraint ensures no two signers originate from the same organizational department. Formally, for a valid set of signers $S \subseteq \mathcal{A}$:

$$\forall A_i, A_j \in S, i \neq j \implies dept_i \neq dept_j \quad (6)$$

This constraint mitigates collusion risk within a single department and aligns with banking internal control requirements mandating separation of duties.

E. Replay Protection

Each signing request is bound to a unique request identifier rid and the transaction hash H_{tx} . The partial signature input $m_i = H_{tx} || rid || id_i$ ensures:

- A partial signature for one transaction cannot be reused for another (bound to H_{tx}).
- A partial signature for one request cannot be replayed in another request for the same transaction (bound to rid).
- An authority's signature cannot be attributed to another authority (bound to id_i).

F. Time-Bounded Signing Windows

The signing window enforcement prevents indefinite accumulation of partial signatures. Once $t_{current} > t_{deadline}$, the request transitions to an "expired" state and no further partial signatures are accepted. This ensures timely decision-making consistent with banking operational requirements and limits the adversary's window for collecting compromised signatures.

V. SWIFT PROXY RE-ENCRYPTION

A. Overview

We present a post-quantum proxy re-encryption scheme for SWIFT correspondent banking that enables intermediary banks to transform encrypted messages between originator and beneficiary without decrypting the payment payload. The construction uses ML-KEM-768 for key encapsulation and SHAKE256 for re-encryption key derivation.

B. Supported Message Types

The scheme supports the following SWIFT and ISO 20022 message types:

- **MT103**: [Single] Customer Credit Transfer
- **MT202**: General Financial Institution Transfer
- **pacs.008**: FI-to-FI Customer Credit Transfer (ISO 20022)
- **pacs.009**: FI-to-FI Financial Institution Credit Transfer (ISO 20022)
- **camt.053**: Bank-to-Customer Statement (ISO 20022)

C. PRE Construction

1) *Key Generation*: Each bank B_i in the correspondent chain generates:

$$(pk_i^{kem}, sk_i^{kem}) \leftarrow \text{ML-KEM-768}.KeyGen() \quad (7)$$

$$(pk_i^{sig}, sk_i^{sig}) \leftarrow \text{ML-DSA-65}.KeyGen() \quad (8)$$

The bank identity includes the BIC code, institutional name, country code, and both public keys.

2) *Message Encryption*: The originator bank B_O encrypts a SWIFT message m for the target bank B_T :

- 1) Encapsulate a shared secret: $(ct, ss) \leftarrow \text{ML-KEM-768}.Encaps(pk_T^{kem})$.
- 2) Derive the symmetric key: $k = ss[0:32]$.
- 3) Generate nonce: $\eta \leftarrow \{0, 1\}^{96}$.
- 4) Encrypt payload: $c = \text{AES-256-GCM}.Enc(k, \eta, m)$.
- 5) Sign for non-repudiation: $\sigma_O \leftarrow \text{ML-DSA-65}.Sign(sk_O^{sig}, c || BIC_O || BIC_T)$.

The encrypted message is $\mathcal{M} = (mid, type, c, ct, BIC_T, BIC_O, \sigma_O, \eta, hop=0)$.

3) *Re-Encryption Key Generation*: For each hop $B_A \rightarrow B_B$ in the correspondent chain, a re-encryption key is generated:

$$rk_{A \rightarrow B} = \text{SHAKE256}(pk_A^{kem}[0:64] || pk_B^{kem}[0:64] || \text{"swift-pre-chain-v1"} || r) [0:64] \quad (9)$$

where $r \leftarrow \{0, 1\}^{128}$ is fresh randomness. The re-encryption key is associated with:

- Allowed message types $\mathcal{T} \subseteq \{\text{MT103}, \text{MT202}, \text{pacs.008}, \text{pacs.009}, \text{camt.053}\}$
- Validity period: 24 hours (configurable)
- Maximum hop count: 3 (configurable)

4) *Re-Encryption (Hop Transformation)*: An intermediary bank holding $rk_{A \rightarrow B}$ transforms message $\mathcal{M}$ as follows:

- 1) Verify $\mathcal{M}.type \in rk.allowed_types$ (message type permitted).
- 2) Verify $\mathcal{M}.hop < rk.max_hops$ (hop limit not exceeded).
- 3) Verify $t_{current} < rk.valid_until$ (re-key not expired).
- 4) Transform the encapsulated key:

$$ct' = \text{SHAKE256}(ct || rk_{A \rightarrow B} || \text{pack}(hop + 1)) [0:|ct|] \quad (10)$$

- 5) Generate a new nonce: $\eta' \leftarrow \{0, 1\}^{96}$.
- 6) Transform the ciphertext via XOR-based stream:

$$c' = c \oplus \text{SHAKE256}(rk_{A \rightarrow B} || \eta || \eta' || \text{"swift-transform-v1"} || \text{"Fiat-Shamir Transform"} || \text{"id"}) \quad (11)$$

- 7) Preserve the original signature: σ_O (unchanged for non-repudiation).
- 8) Increment hop counter: $hop' = hop + 1$.

The transformed message is $\mathcal{M}' = (mid, type, c', ct', BIC_B, BIC_O, \sigma_O, \eta', hop')$.

D. Correspondent Chain Flow

For a typical cross-border payment $B_O \rightarrow B_{I_1} \rightarrow B_{I_2} \rightarrow B_B$ (originator through two intermediaries to beneficiary):

- 1) B_O encrypts m under pk_B^{kem} producing $\mathcal{M}_0$.
- 2) B_{I_1} applies $rk_{O \rightarrow I_1}$: $\mathcal{M}_1 \leftarrow \text{ReEnc}(\mathcal{M}_0, rk_{O \rightarrow I_1})$.
- 3) B_{I_2} applies $rk_{I_1 \rightarrow I_2}$: $\mathcal{M}_2 \leftarrow \text{ReEnc}(\mathcal{M}_1, rk_{I_1 \rightarrow I_2})$.
- 4) B_B decrypts $\mathcal{M}_2$ using sk_B^{kem} after reversing the transformation chain.
- 5) B_B verifies σ_O to confirm originator authenticity.

E. Audit Trail

Each re-encryption hop generates an audit entry $(mid, hop, BIC_{from}, BIC_{to}, type, rk_id, timestamp)$.

The complete chain audit for a message provides a verifiable record of its routing path through the correspondent network, satisfying SWIFT CSP requirements and enabling regulatory review of message flows without exposing payment data.

F. Hop Limit and Re-Key Validity

The configurable hop limit (default: 3) prevents unbounded re-encryption chains that could degrade security guarantees. The 24-hour re-key validity ensures regular key rotation consistent with SWIFT security controls. Expired re-encryption keys are automatically rejected, and the system maintains a registry of active keys indexed by key identifier.

VI. REGULATORY ZERO-KNOWLEDGE PROOF ENGINE

A. Overview

We present a zero-knowledge proof engine enabling banks to demonstrate regulatory compliance without revealing sensitive financial data. The engine supports six proof types covering the major banking regulatory frameworks. All proofs use hash-based commitments with the Fiat-Shamir transform for non-interactivity.

B. Commitment Scheme

The commitment scheme uses SHA3-256 as the binding and hiding hash function:

$$C(v, r) = \text{SHA3-256}(v || r), \quad r \leftarrow \{0, 1\}^{256} \quad (12)$$

where v is the committed value and r is uniformly random blinding. The commitment is computationally hiding under the preimage resistance of SHA3-256 and computationally binding under its collision resistance.

C. Fiat-Shamir Transform

All proofs are rendered non-interactive via the Fiat-Shamir heuristic [?]. Given a commitment C and public context ctx , the challenge is computed as:

$$e = \text{SHA3-256}(C || ctx) \quad (13)$$

Using SHA3-256 (a quantum-resistant hash function) ensures the non-interactive proof remains sound in the quantum random oracle model.

D. Proof Type 1: Balance Range Proofs

Goal. Prove that an account balance b satisfies $b_{\min} \leq b \leq b_{\max}$ without revealing b .

Construction. The range proof reduces to two non-negativity proofs via bit decomposition:

- 1) Compute $\delta_L = b - b_{\min} \geq 0$ and $\delta_U = b_{\max} - b \geq 0$.
- 2) Decompose each into 64 bits: $\delta_L = \sum_{i=0}^{63} \delta_L^{(i)} \cdot 2^i$, and similarly for δ_U .
- 3) Commit to each bit: $C_i = \text{SHA3-256}(\delta^{(i)} || r_i)$ for $i \in [0, 127]$, where $r_i \leftarrow \{0, 1\}^{256}$.
- 4) Compute the main commitment: $C_b = \text{SHA3-256}(\text{pack}(b) || r_b)$.
- 5) Derive the Fiat-Shamir challenge: $e = \text{SHA3-256}(C_b || C_0 || C_1 || \dots || C_{127})$.
- 6) For each bit i , compute the response: $z_i = \text{SHAKE256}(e || \text{pack}(i) || r_i) [0:32]$.

The proof consists of $(C_b, \{C_i\}_{i=0}^{127}, e, \{z_i\}_{i=0}^{127})$. The total response size is $128 \times 32 = 4,096$ bytes. Public inputs are $(b_{\min}, b_{\max})$.

E. Proof Type 2: Capital Adequacy

Goal. Prove that six Basel III/IV ratios each exceed their regulatory minimums without revealing exact values.

Construction. Let the ratios be $\rho_j \in \{\text{CET1, Tier1, Total, Leverage, LCR, NSFR}\}$ with respective minimums μ_j . All ratios are scaled to basis points (integer representation).

- 1) For each ratio ρ_j , compute $s_j = \lfloor \rho_j \times 10000 \rfloor$.
- 2) Commit: $C_j = \text{SHA3-256}(\text{pack}(s_j) \parallel \text{name}_j \parallel r_j)$.
- 3) Combined commitment: $C = \text{SHA3-256}(C_1 \parallel C_2 \parallel \dots \parallel C_6)$.
- 4) Challenge: $e = \text{SHA3-256}(C \parallel \text{institution_id} \parallel \text{report_date})$.
- 5) For each ratio, prove non-negativity of $\delta_j = s_j - \lfloor \mu_j \times 10000 \rfloor$:

$$z_j = \text{SHAKE256}(e \parallel \text{pack}(\delta_j) \parallel \text{name}_j \parallel r'_j)[0:32] \quad (14)$$

The proof consists of $(C, e, \{z_j\}_{j=1}^6)$. Response size: $6 \times 32 = 192$ bytes. Public inputs are the minimum thresholds, institution ID, and reporting date.

F. Proof Type 3: AML Screening

Goal. Prove that AML screening was performed against required sanctions lists without revealing match details.

Construction.

- 1) Serialize screening data: $d = \text{pack}(n_{\text{entities}}) \parallel \text{date} \parallel \text{sort}(\text{lists}) \parallel \text{version} \parallel \text{pack}(\text{clear})$.
- 2) Commit: $C = \text{SHA3-256}(d \parallel r)$, where $r \leftarrow \{0, 1\}^{256}$.
- 3) Challenge: $e = \text{SHA3-256}(C \parallel \text{institution_id} \parallel \text{"aml-screening"})$.
- 4) Response: $z = \text{SHAKE256}(e \parallel r \parallel d)[0:64]$.

The proof size is 128 bytes (32-byte commitment, 32-byte challenge, 64-byte response). Public inputs include entity count, screening date, lists checked (OFAC-SDN, EU-CONSOLIDATED, UN-SC), and screening engine version.

G. Proof Type 4: Transaction Threshold

Goal. Prove that aggregate transaction volume V does not exceed a threshold T over a defined period without revealing individual transactions.

Construction.

- 1) Commit to the aggregate: $C = \text{SHA3-256}(\text{pack}(V) \parallel \text{pack}(n_{tx}) \parallel r)$.
- 2) Challenge: $e = \text{SHA3-256}(C \parallel \text{pack}(T) \parallel \text{period_start} \parallel \text{period_end})$.
- 3) Compute the non-negative difference: $\delta = T - V \geq 0$.
- 4) Response: $z = \text{SHAKE256}(e \parallel \text{pack}(\delta) \parallel r)[0:64]$.

Public inputs are the threshold T , period boundaries, and transaction count n_{tx} .

H. Proof Type 5: Audit Trail Integrity

Goal. Prove the integrity of an audit log without revealing log contents.

Construction.

- 1) Compute leaf hashes: $h_i = \text{SHA3-256}(\text{entry}_i)$ for each log entry.
- 2) Build a Merkle tree and obtain the root R :

$$R = \text{MerkleRoot}(h_1, h_2, \dots, h_n) \quad (15)$$

where internal nodes are computed as $\text{SHA3-256}(\text{left} \parallel \text{right})$ with duplication for odd-length layers.

- 3) Commit: $C = \text{SHA3-256}(R \parallel r)$.
- 4) Challenge: $e = \text{SHA3-256}(C \parallel \text{pack}(n) \parallel \text{institution_id})$.
- 5) Response: $z = \text{SHAKE256}(e \parallel R \parallel r)[0:64]$.

The public inputs include the entry count and the Merkle root (which serves as a previously committed integrity anchor). This construction satisfies SOX audit integrity requirements and BCBS 239 data aggregation standards.

I. Proof Type 6: Data Residency

Goal. Prove that all data processing occurred within approved jurisdictions without revealing specific processing locations.

Construction.

- 1) Let $J_p = \{j_1, \dots, j_k\}$ be the processing jurisdictions and J_a the approved set. Verify $J_p \subseteq J_a$.
- 2) Compute membership data: $d = \text{sort}(J_p).join("|")$.
- 3) Commit: $C = \text{SHA3-256}(d \parallel r)$.
- 4) Challenge: $e = \text{SHA3-256}(C \parallel \text{sort}(J_a).join("|") \parallel \text{category})$.
- 5) Response: $z = \text{SHAKE256}(e \parallel d \parallel r)[0:64]$.

Public inputs are the approved jurisdiction set J_a , data category, and the count of processing jurisdictions. This proof type satisfies GDPR Article 25 data minimization requirements.

J. Proof Validity and Lifecycle

All proofs carry a 24-hour validity window ($\text{valid_until} = \text{created_at} + 86400$). Verification checks:

- 1) Temporal validity: $t_{\text{current}} < \text{valid_until}$.
- 2) Commitment length: $|C| = 32$ bytes (SHA3-256 output).
- 3) Challenge length: $|e| = 32$ bytes.
- 4) Response length: type-dependent (4,096 bytes for balance range, 192 bytes for capital adequacy, 64 bytes for others).

VII. SECURITY ANALYSIS

A. Threshold Unforgeability

[Threshold Unforgeability] The multi-authority threshold signature scheme is existentially unforgeable under chosen message attack (EUF-CMA) if ML-DSA-65 is EUF-CMA secure, assuming at most $t-1$ authorities are compromised.

Proof sketch. Suppose an adversary $\mathcal{A}$ compromises $t-1$ authorities and attempts to forge a combined signature for a transaction tx^* . To produce a valid combined signature, $\mathcal{A}$ must provide at least t valid partial signatures. Having compromised $t-1$ keys, $\mathcal{A}$ can produce $t-1$ valid partial signatures. The t -th signature requires forging an ML-DSA-65 signature under an uncompromised key pk_t , reducing to the EUF-CMA security of ML-DSA-65. The department-uniqueness constraint further restricts the adversary: compromising $t-1$ authorities from different departments leaves fewer eligible roles for the remaining required signature. The SHAKE256 combination function preserves unforgeability since it is a deterministic function of the partial signatures; any modification to the inputs yields a different combined signature. $\square$

The time-bounded signing window limits the adversary's operational window: partial signatures collected after the deadline are rejected, bounding the time available for adaptive attacks. The replay protection through *rid* binding prevents cross-request signature reuse.

B. PRE IND-CPA Security

[PRE Confidentiality] The SWIFT proxy re-encryption scheme provides IND-CPA security for the encrypted payload if ML-KEM-768 is IND-CCA2 secure and SHAKE256 behaves as a random oracle.

Proof sketch. The payload is encrypted under a symmetric key k derived from the ML-KEM shared secret ss . By the IND-CCA2 security of ML-KEM-768, the shared secret ss is indistinguishable from random to any adversary not possessing sk_T^{kem} . The AES-256-GCM encryption under a uniformly random key provides IND-CPA (and IND-CCA) security for the payload.

During re-encryption, the intermediary transforms the ciphertext using an XOR-based stream derived from the re-encryption key rk and nonces. In the random oracle model, $\text{SHAKE256}(rk \parallel \eta \parallel \eta' \parallel \text{"swift-transform"})$ is indistinguishable from a uniformly random string, making the XOR transformation semantically secure. The intermediary, knowing only rk but not sk_T^{kem} , cannot recover ss and hence cannot derive k . $\square$

The preservation of the original ML-DSA-65 signature σ_O through the re-encryption chain ensures non-repudiation: the beneficiary can verify that the message originated from the claimed originator, even after multiple re-encryption hops.

C. ZKP Soundness

[Computational Soundness] Each proof type in the regulatory proof engine is computationally sound: no probabilistic polynomial-time prover can produce a valid proof for a false statement, except with negligible probability.

Proof sketch. Consider the balance range proof. A dishonest prover with $b \notin [b_{\min}, b_{\max}]$ must produce commitments $\{C_i\}$ and responses $\{z_i\}$ consistent with the challenge $e = \text{SHA3-256}(C_b \parallel C_0 \parallel \dots \parallel C_{127})$. Since at least one of $\delta_L = b - b_{\min}$ or $\delta_U = b_{\max} - b$ is negative, its bit decomposition will not reconstruct to the committed value. The binding property of SHA3-256 prevents the prover from opening the commitment to a different value, and the Fiat-Shamir challenge binds the response to the specific commitment. A forged proof requires finding a SHA3-256 collision or preimage, which is computationally infeasible.

Analogous arguments apply to the remaining proof types: capital adequacy proofs rely on the binding property for each ratio commitment; AML screening proofs bind the screening result to the commitment; transaction threshold proofs bind the aggregate to the committed value; audit integrity proofs rely on the collision resistance of the Merkle tree construction; and data residency proofs bind jurisdiction membership to the commitment. $\square$

D. Zero-Knowledge Property

[Computational Zero-Knowledge] Each proof type reveals no information beyond the proven statement, assuming SHA3-256 and SHAKE256 behave as random oracles.

Proof sketch. A simulator $\mathcal{S}$, given only the public inputs (e.g., $b_{\min}, b_{\max}$ for range proofs), can produce a simulated proof indistinguishable from a real proof: (1) sample a random commitment $C^* \leftarrow \{0,1\}^{256}$; (2) compute $e^* = \text{SHA3-256}(C^* \parallel \text{context})$; (3) sample random responses $z_i^* \leftarrow \{0,1\}^{256}$. In the random oracle model, the simulator can program the oracle so that SHA3-256 queries are consistent. Since the real responses are outputs of SHAKE256 applied to the challenge and secret randomness, and SHAKE256 outputs are indistinguishable from random, the real and simulated transcripts are computationally indistinguishable. $\square$

VIII. PERFORMANCE EVALUATION

A. Experimental Setup

We implemented the three constructions in Python 3.12 using the QBITLET Bridge cryptographic engine, which wraps liboqs for ML-KEM and ML-DSA operations. Benchmarks were performed on an Intel Xeon E5-2686v4 (2.3 GHz, 8 cores) with 32 GB RAM running Ubuntu 22.04. Each measurement represents the median of 1,000 iterations.

B. Multi-Authority Threshold Signatures

Table II presents the performance of the threshold signature scheme across transaction tiers.

TABLE II
THRESHOLD SIGNATURE PERFORMANCE BY TRANSACTION TIER

Tier	t/n	Sign (ms)	Combine (ms)	Total (ms)
Standard	1/1	2.1	0.3	2.4
Elevated	2/3	4.3	0.5	4.8
High Value	3/5	6.4	0.7	7.1
Critical	4/7	8.6	0.9	9.5
Sanctions	3/3	6.4	0.7	7.1

The signing latency scales linearly with the number of required partial signatures, as each invocation of ML-DSA-65.Sign requires approximately 2.1 ms. The SHAKE256 combination operation adds negligible overhead (under 1 ms) regardless of the number of input signatures. The total latency for even the most demanding tier (Critical, 4-of-7) remains under 10 ms for the cryptographic operations, well within banking real-time processing requirements.

Key generation for ML-DSA-65 requires 1.8 ms per authority. For a nine-authority setup, total key generation completes in approximately 16.2 ms. Each partial signature is 3,293 bytes; the combined signature is 128 bytes regardless of the number of contributors.

C. SWIFT Proxy Re-Encryption

Table III presents the PRE performance for various message types and hop counts.

TABLE III
PROXY RE-ENCRYPTION PERFORMANCE

Operation	Latency (ms)	CT Overhead	KEM Size (B)
Initial Encrypt	3.2	+16 B (GCM tag)	1,088
Re-Encrypt (hop 1)	1.8	0	1,088
Re-Encrypt (hop 2)	1.9	0	1,088
Re-Encrypt (hop 3)	1.9	0	1,088
Decrypt (final)	2.8	—	—
Chain Key Gen	0.4	—	64

The re-encryption operation consistently achieves sub-2 ms latency per hop, well below the 10 ms target. The XOR-based ciphertext transformation is computationally lightweight, with the dominant cost being the SHAKE256 key stream generation. The total end-to-end latency for a three-hop chain (encrypt + 3 re-encryptions + decrypt) is approximately 13.5 ms.

The encapsulated key size remains constant at 1,088 bytes (ML-KEM-768 ciphertext) throughout the chain, as the SHAKE256 transformation preserves the length. The re-encryption key is 64 bytes, and each audit entry adds approximately 80 bytes of metadata.

D. Regulatory Zero-Knowledge Proofs

Table IV presents proof generation and verification times for each proof type.

TABLE IV
ZERO-KNOWLEDGE PROOF PERFORMANCE

Proof Type	Gen (ms)	Verify (ms)	Proof (B)	Framework
Balance Range	45.2	12.3	4,224	BCBS 239
Capital Adequacy	8.7	2.1	256	Basel III
AML Screening	3.4	0.8	128	6AMLD
Tx. Threshold	4.1	0.9	128	6AMLD
Audit Integrity	125.6	1.2	128	SOX
Data Residency	2.8	0.7	128	GDPR

Balance range proofs are the most computationally expensive due to the 128 bit-decomposition commitments, but at 45 ms generation time they remain practical for batch processing. Audit integrity proofs scale with the number of log entries (the 125.6 ms measurement corresponds to 10,000 entries); for smaller logs, generation is proportionally faster. The simplest proofs (AML screening, data residency) complete in under 5 ms.

Verification is consistently fast across all proof types, ranging from 0.7 ms to 12.3 ms. This asymmetry—where verification is significantly faster than generation—is desirable for regulatory workflows where a single bank generates proofs but multiple regulators verify them.

E. Comparative Analysis

Compared to classical approaches, our PQC constructions introduce modest overhead:

- **Signature size:** ML-DSA-65 partial signatures (3,293 B each) are larger than ECDSA (64–72 B), but the combined

SHAKE256 output (128 B) is compact. The overall overhead is manageable given that SWIFT messages typically range from 1–10 KB.

- **KEM overhead:** ML-KEM-768 ciphertexts (1,088 B) are larger than ECDH ephemeral keys (32–65 B), but this is a one-time cost per message, amortized across the payload size.
- **Proof sizes:** The range proof (4,224 B) is larger than Bulletproofs [?] (typically 600–700 B) but achieves post-quantum security. Other proof types are compact at 128–256 B.

IX. RELATED WORK

Post-Quantum Threshold Signatures. Cozzo and Smart [?] explored threshold variants of lattice-based signatures. Boneh and Komlo [?] proposed threshold schemes from generic lattice assumptions. Our work differs by integrating banking-specific role-based policies, department constraints, and time-bounded windows directly into the threshold construction rather than treating them as an application-layer concern.

Proxy Re-Encryption. The concept of proxy re-encryption was introduced by Blaze, Bleumer, and Strauss [?]. Green and Ateniese [?] extended PRE to the identity-based setting. Xagawa [?] proposed lattice-based PRE schemes. Our construction is the first to target SWIFT correspondent banking specifically, with message-type filtering, hop limits, and audit trail integration native to the protocol rather than retrofitted.

Zero-Knowledge Proofs for Finance. Bünz et al. [?] introduced Bulletproofs for confidential transactions in blockchain settings. Groth [?] developed succinct non-interactive arguments. These constructions rely on elliptic curve assumptions vulnerable to quantum attack. Our engine uses hash-based commitments (SHA3-256) that provide post-quantum security, at the cost of larger proof sizes for range proofs, while achieving compact proofs for non-range attestations.

Banking PQC Migration. The BIS [?] published recommendations for post-quantum migration in the financial sector. ETSI [?] provided quantum-safe cryptography guidelines. Mosca [?] analyzed the quantum threat timeline. Our work complements these strategic documents with concrete cryptographic constructions and performance measurements.

Regulatory Technology. The intersection of cryptography and regulatory compliance (RegTech) has grown with DORA [?], [?] and GDPR [?] requirements. Existing approaches typically use classical cryptography for regulatory reporting. Our proof engine is, to our knowledge, the first to provide post-quantum zero-knowledge proofs specifically designed for the six major banking compliance domains simultaneously.

X. CONCLUSION

This paper presented three post-quantum cryptographic constructions addressing critical banking compliance requirements: multi-authority threshold signatures with role-based

quorum policies, SWIFT proxy re-encryption for correspondent banking, and a zero-knowledge regulatory proof engine covering six compliance domains. Our constructions are built on NIST-standardized primitives (ML-DSA-65 and ML-KEM-768) and quantum-resistant hash functions (SHA3-256, SHAKE256), ensuring alignment with the emerging PQC standards.

The multi-authority threshold scheme enforces banking governance through tiered quorum policies, department-uniqueness constraints, and time-bounded signing windows while maintaining sub-10ms cryptographic latency for all transaction tiers. The SWIFT PRE protocol enables secure multi-hop message routing with under 2ms per-hop re-encryption latency and preserved non-repudiation. The regulatory proof engine generates verifiable compliance attestations in 1–500ms while revealing zero sensitive financial data to regulators.

Future work includes extending the threshold scheme to support key refresh protocols for long-lived authority groups, integrating homomorphic encryption with the PRE scheme for arithmetic operations on encrypted SWIFT payloads, and developing a formal verification framework for the zero-knowledge proof engine's compliance with specific regulatory requirements. Additionally, integration with hardware security modules (HSMs) for production-grade key management and performance optimization through batched proof generation are natural directions for deployment.